Sesión 11 — Variables, tipos, operadores y condicionales en JavaScript

Enfoque de la sesión

En esta sesión vas a construir la base operativa del lenguaje. El objetivo no es memorizar una lista de conceptos aislados, sino empezar a escribir código que **guarde información**, **tome decisiones** y **produzca resultados útiles**. Todo lo que trabajes aquí aparecerá más adelante cuando tengas que transformar datos, reaccionar a eventos, validar formularios o renderizar interfaces.

1. Introducción

JavaScript es el lenguaje que te permite añadir comportamiento a una aplicación web. Gracias a él puedes comprobar datos, decidir qué debe ocurrir en cada caso, modificar información y responder a las acciones de una persona usuaria.

Antes de trabajar con arrays, DOM, eventos o APIs, necesitas dominar un núcleo fundamental del lenguaje. Ese núcleo está formado por:

- variables;
- tipos de datos;
- operadores;
- comparaciones;
- condicionales.

Estos elementos son la base de casi cualquier programa. Cada vez que una aplicación decide si una persona puede acceder, si un producto tiene descuento, si un formulario es válido o si un mensaje debe mostrarse de una forma u otra, está utilizando estas ideas.

En esta sesión no vas a estudiar JavaScript como una lista abstracta de definiciones. Vas a estudiarlo como una herramienta para resolver problemas sencillos pero reales.

---

2. Variables y almacenamiento de información

2.1. Qué es una variable

Una variable es un identificador asociado a un valor. Te permite almacenar información para utilizarla después en tu programa.

Observa este ejemplo:

```
const nombre = "Lucía";
const edad = 22;
```

Aquí se están guardando dos datos: un nombre y una edad. A partir de ese momento puedes reutilizarlos en otras operaciones.

```
const nombre = "Lucía";
const edad = 22;

console.log(nombre);
console.log(edad);
```

> **Idea clave**
>
> Programar consiste en gran parte en **trabajar con datos**. Para ello necesitas poder **nombrarlos**, **guardarlos** y **utilizarlos** con claridad.

2.2. Declaración de variables: `let`, `const` y `var`

En JavaScript existen tres formas tradicionales de declarar variables:

- `let`
- `const`
- `var`

En el desarrollo actual, las formas principales son `let` y `const`.

`const`

Se utiliza cuando no vas a reasignar la variable.

```
const precioBase = 100;
```

No puedes hacer esto después:

```
const precioBase = 100;
```

```
precioBase = 120; // Error
```

`let`

Se utiliza cuando sí necesitas reasignar el valor.

```
let cantidad = 1;
cantidad = 2;
```

`var`

Existe por compatibilidad con código antiguo, pero hoy no es la opción recomendada para escribir código nuevo en la mayoría de casos de enseñanza inicial, porque su comportamiento con ámbito y hoisting genera más confusión.

```
var ciudad = "Madrid";
```

Regla práctica para esta sesión

Usa `const` por defecto.
Usa `let` cuando sepas que el valor va a cambiar.
Evita `var` salvo que necesites entender código heredado.

2.3. Nombrado de variables

Los nombres de variables deben ayudarte a entender el programa. Un nombre confuso convierte un ejercicio sencillo en uno difícil de leer.

Mal ejemplo

```
const x = 19;
const y = true;
```

Mejor ejemplo

```
const edadUsuario = 19;
const accesoPermitido = true;
```

2.4. Buenas prácticas iniciales

- usa nombres significativos;

- evita abreviaturas oscuras;
- intenta que el nombre refleje la función del dato;
- no reutilices una misma variable para significados distintos.

Referencias específicas del subtema

- MDN Web Docs — `const`
- MDN Web Docs — `let`
- MDN Web Docs — Grammar and types

---

3. Tipos de datos básicos

3.1. Qué es un tipo de dato

El tipo de dato indica qué clase de valor estás manejando. JavaScript necesita distinguir si un valor es un número, un texto, un booleano o la ausencia de un valor útil.

Los tipos básicos más importantes al comenzar son:

- `string`
- `number`
- `boolean`
- `undefined`
- `null`

3.2. `string`

Representa texto.

```
const nombre = "Aitana";
const mensaje = 'Hola';
```

3.3. `number`

Representa valores numéricos.

```
const edad = 30;
const precio = 19.99;
```

3.4. `boolean`

Representa un valor lógico: verdadero o falso.

```
const esMayorDeEdad = true;
const productoDisponible = false;
```

3.5. `undefined`

Aparece cuando una variable existe pero todavía no tiene valor asignado.

```
let resultado;
console.log(resultado); // undefined
```

3.6. `null`

Representa de forma intencional la ausencia de valor.

```
const usuarioSeleccionado = null;
```

> ⚠ **`null` y `undefined` no significan exactamente lo mismo**
>
> Aunque a veces puedan parecer similares, conviene distinguirlos:
>
> - `undefined` : no se ha asignado valor.
> - `null` : se ha indicado explícitamente que no hay valor.

3.7. Comprobar tipos con `typeof`

Puedes inspeccionar el tipo de un valor con `typeof` .

```
const nombre = "Ana";
const edad = 28;
const activo = true;

console.log(typeof nombre); // string
console.log(typeof edad);   // number
console.log(typeof activo); // boolean
```

Referencias específicas del subtema

- MDN Web Docs — JavaScript data types and data structures
- MDN Web Docs — `typeof`

4. Salida por consola y template literals

4.1. `console.log()`

Durante el aprendizaje y también en el trabajo profesional, la consola es una herramienta básica para observar valores y depurar.

```
const nombre = "Elena";
console.log(nombre);
```

También puedes combinar texto y variables.

```
const nombre = "Elena";
const edad = 26;

console.log("Nombre:", nombre);
console.log("Edad:", edad);
```

4.2. Template literals

Los template literals permiten construir cadenas de forma más clara utilizando acentos graves.

```
const nombre = "Elena";
const edad = 26;

console.log(`Hola, me llamo ${nombre} y tengo ${edad} años.`);
```

Esto mejora mucho la legibilidad frente a concatenaciones más antiguas.

Referencias específicas del subtema

- MDN Web Docs — `console`
- MDN Web Docs — Template literals

---

5. Operadores

5.1. Operadores aritméticos

Se utilizan para realizar cálculos.

Operador	Significado	Ejemplo
`+`	suma	`5 + 3`
`-`	resta	`8 - 2`
`*`	multiplicación	`4 * 6`
`/`	división	`20 / 5`
`%`	resto	`10 % 3`

```
const precio = 50;
const descuento = 10;
const precioFinal = precio - descuento;

console.log(precioFinal);
```

5.2. Operadores de comparación

Permiten comparar valores y devuelven `true` o `false`.

Operador	Significado	Ejemplo
`>`	mayor que	`edad > 18`
`<`	menor que	`edad < 18`
`>=`	mayor o igual	`nota >= 5`
`<=`	menor o igual	`stock <= 0`
`===`	igualdad estricta	`rol === "admin"`
`!==`	desigualdad estricta	`estado !== "cancelado"`

5.3. Operadores lógicos

Se usan para combinar condiciones.

Operador	Significado	Ejemplo
`&&`	y	`edad >= 18 && tieneEntrada`

Operador	Significado	Ejemplo
`||`	o	`esAdmin || esEditor`
`!`	negación	`!estaBloqueado`

```
const edad = 20;
const tieneEntrada = true;

console.log(edad >= 18 && tieneEntrada); // true
```

5.4. Operador de asignación

El operador `=` asigna un valor.

```
let cantidad = 3;
```

No debes confundirlo con la comparación.

- `=` asigna
- `===` compara estrictamente

> **Error muy frecuente**
>
> Confundir asignación con comparación produce errores lógicos importantes.
> Cuando quieras comprobar igualdad, tu referencia habitual debe ser `===`.

Referencias específicas del subtema

- MDN Web Docs — Expressions and operators
- MDN Web Docs — Arithmetic operators
- MDN Web Docs — Comparison operators
- MDN Web Docs — Logical operators

---

6. Coerción: conversiones implícitas y explícitas

6.1. Qué es la coerción

JavaScript puede convertir valores de un tipo a otro automáticamente en ciertas operaciones. A esto se le llama coerción.

Ejemplo:

```
console.log("5" + 2); // "52"
```

Aquí no se realiza una suma numérica, sino una concatenación de texto.

Otro ejemplo:

```
console.log("5" - 2); // 3
```

En este caso JavaScript convierte `"5"` a número.

6.2. Por qué puede generar errores

La coerción automática puede producir resultados sorprendentes si no controlas los tipos de los valores.

```
const precio = "100";
const recargo = 20;

console.log(precio + recargo); // "10020"
```

Si querías un cálculo numérico, ese resultado es incorrecto.

6.3. Conversión explícita

En muchos casos conviene convertir los datos de forma clara.

```
const precio = Number("100");
const recargo = 20;

console.log(precio + recargo); // 120
```

O también:

```
const cantidad = parseInt("42", 10);
console.log(cantidad);
```

Buena práctica

Cuando una operación depende realmente de trabajar con números, intenta asegurarte de que los valores son numéricos antes de operar con ellos.

Referencias específicas del subtema

- MDN Web Docs — Type coercion
- MDN Web Docs — `Number()`
- MDN Web Docs — `parseInt()`

---

7. Comparación estricta y comparación no estricta

7.1. `===` y `!==`

La comparación estricta compara tanto el valor como el tipo.

```
console.log(5 === 5);     // true
console.log(5 === "5");   // false
```

La desigualdad estricta funciona del mismo modo:

```
console.log(5 !== "5"); // true
```

7.2. `==` y `!=`

Existen operadores no estrictos, pero realizan coerción automática y pueden producir resultados poco intuitivos.

```
console.log(5 == "5"); // true
```

Aunque esto pueda parecer útil en algunos casos, para el aprendizaje y para la mayoría de escenarios cotidianos conviene evitarlo.

Norma de trabajo recomendada

Utiliza `===` y `!==` como referencia habitual.

Referencias específicas del subtema

- MDN Web Docs — Equality comparisons and sameness

---

8. Condicionales

8.1. `if`

Permite ejecutar código solo si una condición se cumple.

```
const edad = 20;

if (edad >= 18) {
  console.log("Puedes acceder.");
}
```

8.2. `if ... else`

Permite decidir entre dos caminos.

```
const edad = 16;

if (edad >= 18) {
  console.log("Puedes acceder.");
} else {
  console.log("No puedes acceder.");
}
```

8.3. `if ... else if ... else`

Sirve para evaluar varios casos.

```
const nota = 8;

if (nota < 5) {
  console.log("Suspenso");
} else if (nota < 7) {
  console.log("Aprobado");
} else if (nota < 9) {
  console.log("Notable");
} else {
  console.log("Sobresaliente");
}
```

8.4. Operador ternario

Es una forma compacta de expresar decisiones simples.

```
const edad = 20;
const mensaje = edad >= 18 ? "Mayor de edad" : "Menor de edad";

console.log(mensaje);
```

Debe utilizarse cuando realmente mejora la legibilidad. Si la condición es compleja, suele ser mejor usar `if`.

8.5. `switch`

Es útil cuando quieres comparar el mismo valor frente a varios casos concretos.

```
const dia = "lunes";

switch (dia) {
  case "lunes":
    console.log("Inicio de semana");
    break;
  case "viernes":
    console.log("Fin de semana cerca");
    break;
  default:
    console.log("Día intermedio");
}
```

> **Criterio de uso**
>
> `switch` no es “mejor” que `if` por sí mismo.
> Te conviene usarlo cuando estás evaluando **muchos casos distintos del mismo valor** y mejora la claridad del código.

Referencias específicas del subtema

- MDN Web Docs — `if...else`
- MDN Web Docs — Conditional (ternary) operator
- MDN Web Docs — `switch`

---

9. Análisis técnico avanzado

9.1. `const` no vuelve inmutable el valor

`const` impide reasignar la variable, pero eso no significa que el contenido sea inmutable si se trata de un objeto o un array.

```
const usuario = { nombre: "Ana" };
usuario.nombre = "Laura";

console.log(usuario.nombre); // "Laura"
```

No puedes hacer esto:

```
const usuario = { nombre: "Ana" };
usuario = { nombre: "Laura" }; // Error
```

Pero sí puedes modificar propiedades internas si el valor es un objeto.

9.2. El tipado dinámico aporta flexibilidad, pero exige atención

En JavaScript una variable puede contener distintos tipos a lo largo del tiempo.

```
let dato = 42;
dato = "ahora soy un texto";
```

Esta flexibilidad puede ser cómoda, pero también puede introducir errores si no mantienes disciplina al diseñar tu código.

9.3. No toda expresión verdadera o falsa es un booleano explícito

JavaScript considera algunos valores como “truthy” y otros como “falsy”. Esto influye en las condiciones.

Se consideran falsy, entre otros:

- `false`
- `0`
- `""`
- `null`
- `undefined`
- `NaN`

Ejemplo:

```
const nombre = "";

if (nombre) {
  console.log("Hay nombre");
} else {
  console.log("No hay nombre");
}
```

Esto es útil, pero también puede ocultar errores si no tienes claro qué valor estás comprobando realmente.

9.4. Legibilidad frente a brevedad

Un programa corto no siempre es un programa claro. En una fase inicial debes priorizar:

- nombres explícitos;
- condiciones comprensibles;
- pasos separados;
- salida verificable por consola.

La legibilidad no es un lujo: es una práctica profesional.

9.5. La consola no es solo para “ver si funciona”

La consola te sirve para inspeccionar:

- qué valores tienes en cada paso;
- qué tipo de dato estás manejando;
- qué condición se está cumpliendo;
- en qué parte del flujo aparece un error lógico.

Referencias específicas del subtema

- MDN Web Docs — `const`
- MDN Web Docs — Truthy
- MDN Web Docs — Falsy
- MDN Web Docs — Debugging JavaScript

---

10. Ejemplo integrador de sesión

A continuación tienes un ejemplo que reúne varios de los conceptos trabajados.

```
const nombreUsuario = "Marta";
const edadUsuario = 19;
const tieneCupon = true;
const precioBase = 50;

let precioFinal = precioBase;

if (edadUsuario < 18) {
  console.log(`Lo sentimos, ${nombreUsuario}, no puedes continuar con la
compra.`);
} else {
  if (tieneCupon) {
    precioFinal = precioBase - 10;
  }

  const mensajePrecio =
    precioFinal < precioBase
      ? `Se ha aplicado un descuento. Precio final: ${precioFinal} €`
      : `No se ha aplicado descuento. Precio final: ${precioFinal} €`;

  console.log(`Hola, ${nombreUsuario}.`);
  console.log(mensajePrecio);
}
```

Qué se está aplicando aquí

Elemento	Uso en el ejemplo
Variables	almacenamiento de nombre, edad, cupón y precio
Tipos	`string`, `number`, `boolean`
Operadores	comparación, resta y asignación
Condicionales	control del flujo según edad y cupón
Template literals	generación de mensajes más legibles
`let` y `const`	diferencia entre valor fijo y valor que cambia

---

11. Errores frecuentes al comenzar

⚠ **Errores muy habituales**

Estos fallos aparecen con mucha frecuencia al empezar. Detectarlos pronto te ayudará a avanzar con más seguridad.

11.1. Usar `=` cuando querías comparar

```
let edad = 18;

// Incorrecto conceptual en una condición si lo que querías era comparar
if (edad = 18) {
  console.log("Esto no significa lo que crees");
}
```

11.2. Confiar en que un número escrito como texto se comporte siempre como número

```
const cantidad = "10";
const total = cantidad + 5;

console.log(total); // "105"
```

11.3. Encadenar condiciones poco legibles

```
const edad = 20;
const tieneEntrada = true;
const estaRegistrado = true;

if (edad >= 18 && tieneEntrada && estaRegistrado) {
  console.log("Acceso permitido");
}
```

Este código puede ser correcto, pero si la lógica crece demasiado quizá necesites separar comprobaciones en variables más expresivas.

```
const esMayorDeEdad = edad >= 18;
const accesoPermitido = esMayorDeEdad && tieneEntrada && estaRegistrado;

if (accesoPermitido) {
```

```
  console.log("Acceso permitido");
}
```

11.4. Abusar del ternario cuando la lógica ya no es simple

El operador ternario es útil, pero no debe sustituir a un `if` claro cuando la condición o el resultado se vuelven complejos.

---

12. Síntesis de la sesión

En esta sesión has trabajado los primeros pilares del lenguaje:

- cómo almacenar información;
- qué tipos de datos básicos existen;
- cómo operar con ellos;
- cómo comparar valores;
- cómo tomar decisiones en función de condiciones;
- cómo empezar a depurar desde consola.

Este núcleo es imprescindible porque las siguientes sesiones se apoyarán constantemente en él. Muy pronto necesitarás estas mismas ideas para recorrer colecciones, encapsular lógica en funciones, manipular datos más complejos y construir comportamiento en el navegador.

✓ Al terminar esta sesión deberías ser capaz de...

- declarar variables con `let` y `const` con criterio;
- distinguir los tipos básicos más importantes;
- utilizar operadores aritméticos, lógicos y de comparación;
- detectar problemas frecuentes de coerción;
- escribir condiciones claras con `if`, `else`, ternario y `switch`;
- inspeccionar valores básicos mediante `console.log()`.

---

13. Referencias documentales generales

Referencias oficiales y de consulta

- MDN Web Docs — JavaScript Guide

- MDN Web Docs — Grammar and types
- MDN Web Docs — Expressions and operators
- MDN Web Docs — `if...else`
- MDN Web Docs — `switch`
- MDN Web Docs — Template literals
- MDN Web Docs — Equality comparisons and sameness
- MDN Web Docs — JavaScript data types and data structures

Recomendación de estudio

Al repasar esta sesión, no te limites a releer ejemplos. Reescribe el código, cambia valores, provoca errores intencionadamente y observa qué ocurre. En JavaScript, comprender por qué una salida es distinta de la esperada es una parte fundamental del aprendizaje.